

Awesom Living

Zigbee Device Disconnection

All possible reasons a Zigbee sensor can go offline — and how to fix each one

Date: September 2026 | Version: 1.0

1. Our Zigbee Setup

Before diving into the disconnection reasons, here is what our Zigbee network looks like in the Awesom Living pilot home:

Component	Details
Coordinator	SONOFF ZBDongle-E (EFR32MG21 chip) → plugged into Raspberry Pi USB 3.0 port
Zigbee Software	Zigbee2MQTT running on Raspberry Pi 4 (IP: 192.168.1.14)
MQTT Broker	Mosquitto (local, on same Pi)
WiFi Router	TP-Link Deco M4 (2.4 GHz + 5 GHz, WPA2 Personal)
Zigbee Devices (6)	Motion Sensor (room_motion), Presence Sensor (bathroom_presence), Door Sensors ×3 (main_gate, bedroom_window, kitchen_window), Emergency Button (safecall_button)
Network Type	Star topology — all 6 devices connect directly to coordinator (no routers/repeaters in the mesh)

Important: All 6 of our Zigbee devices are battery-powered "sleepy end devices." They spend most of their time asleep to save battery and only wake up briefly to send data or check in with the coordinator. This is important because many disconnection causes are specific to this type of device.

2. WiFi and Zigbee Radio Interference

Likelihood: High

What happens

Zigbee and WiFi both operate on the 2.4 GHz radio band. When the WiFi router and the Zigbee coordinator are using overlapping frequencies, WiFi signals (which are much stronger) drown out Zigbee signals, causing devices to lose connection or fail to send data.

How this applies to us

Our TP-Link Deco M4 router broadcasts on 2.4 GHz. Our SONOFF ZBDongle-E coordinator also uses 2.4 GHz. Both are physically close to each other (connected through the same PoE switch and D-Link switch). If the WiFi channel and Zigbee channel overlap, interference is almost guaranteed.

Zigbee uses channels 11–26. WiFi uses channels 1–11. The overlap looks like this:

- WiFi channel 1 overlaps with Zigbee channels 11–14.
- WiFi channel 6 overlaps with Zigbee channels 15–19.
- WiFi channel 11 overlaps with Zigbee channels 20–24.
- Zigbee channels 25 and 26 have the least overlap with any WiFi channel.

Our fix: Set Zigbee to channel 25 in Zigbee2MQTT configuration. This minimizes overlap with any WiFi channel. After changing the channel, all devices need to be re-paired.

Official references

- [Zigbee2MQTT — Configuration Reference](#)
- [Oscium — ZigBee and Wi-Fi Coexistence](#)
- [Silicon Labs — Zigbee and WiFi Coexistence \(AN1017\)](#)

3. USB 3.0 Port Interference

Likelihood: High

What happens

USB 3.0 ports generate radio frequency interference in the 2.4 GHz range — the exact same band Zigbee uses. When a Zigbee USB dongle is plugged directly into a USB 3.0 port, the USB 3.0 shielding leaks enough noise to severely degrade the Zigbee radio signal. Devices that were working fine suddenly become unreliable or drop off completely.

How this applies to us

Our SONOFF ZBDongle-E is currently plugged directly into the Raspberry Pi 4's USB 3.0 port. The Pi 4 has both USB 2.0 and USB 3.0 ports. This is a known and well-documented issue that affects many Zigbee setups.

Our fix: Use a USB 2.0 extension cable (1–2 meters) to physically move the dongle away from the Pi. This does two things: moves the dongle away from the USB 3.0 interference, and gives the antenna some distance from other electronics. Alternatively, plug the dongle into a USB 2.0 port on the Pi (the black ports, not the blue ones).

Official references

- [Zigbee2MQTT — Adapter FAQ \(USB Extension Cable\)](#)
- [Intel — USB 3.0 Radio Frequency Interference Impact on 2.4 GHz Devices](#)

4. Battery Running Low or Dead

Likelihood: High

What happens

All battery-powered Zigbee devices eventually run out of battery. As the battery voltage drops, the radio signal gets weaker — the device can still technically send data, but it can no longer reach the coordinator reliably. Eventually, it stops transmitting altogether and appears "offline."

What makes this tricky is that the signal degrades gradually. You might see intermittent disconnections for days or weeks before the device goes completely silent.

How this applies to us

All 6 of our Zigbee devices are battery-powered:

- Motion Sensor — CR2450 coin cell battery.
- Presence Sensor — typically 2× CR123A or USB-rechargeable.
- Door/Window Sensors (×3) — CR2032 coin cell battery.
- Emergency Button — CR2032 coin cell battery.

SONOFF Zigbee sensors typically report battery percentage in their Zigbee payload. Zigbee2MQTT publishes this as a "battery" field in the MQTT message.

Our fix: Monitor the battery percentage from Zigbee2MQTT MQTT messages. Set up an alert in Node-RED when battery drops below 20%. Replace batteries proactively before they die. Typical battery life for SONOFF sensors: 1–2 years depending on reporting frequency.

Official references

- [Zigbee2MQTT — Device Availability Tracking](#)
- [Silicon Labs — Sleepy End Devices \(Battery Behavior\)](#)

5. Device Too Far from Coordinator (Range)

Likelihood: Medium

What happens

Zigbee has a limited radio range. In open air, SONOFF sensors can reach about 30–40 meters. But indoors, walls, floors, furniture, metal surfaces, and mirrors absorb or reflect the signal. Each wall the signal has to pass through reduces range by roughly 25–50%. After 2–3 solid walls, the signal may be too weak to maintain a reliable connection.

How this applies to us

In the pilot home, the coordinator (Pi + ZBDongle-E) is in one location. If any sensor (especially the main_gate door sensor, which is likely near the front door / far from the Pi) is more than 2–3 walls away, signal strength could be marginal.

Our network has no Zigbee routers (repeaters) in the mesh. All 6 devices connect directly to the coordinator in a star pattern. This means every device must be within direct range of the coordinator — there is no "hop" through intermediate devices.

Our fix: If a specific device keeps dropping, add a Zigbee router (any mains-powered Zigbee device, such as a smart plug) between the coordinator and the distant sensor. This creates a relay point. Also consider relocating the Pi/dongle to a more central location in the home.

Official references

- [Silicon Labs — Zigbee Mesh Networking Concepts](#)
- [Zigbee2MQTT — Network Map](#)

6. Coordinator Crash or Restart

Likelihood: Medium

What happens

When the Zigbee coordinator (the USB dongle + Zigbee2MQTT software) crashes, restarts, or loses power, the entire Zigbee network goes down temporarily. Mains-powered devices (routers) usually rejoin automatically. But battery-powered "sleepy" devices are a different story — they check in with the coordinator on a schedule (every few minutes to every few hours). If the coordinator was down during their check-in window, they may not realize it is back up and stay "offline" until their next check-in cycle.

How this applies to us

Our coordinator runs on the Raspberry Pi 4. The Pi can restart for several reasons:

- Power outage or unstable power supply (must be 5V 3A USB-C, not a phone charger).
- SD card corruption (common on Pi's — writes to SD cards can fail over time).
- Zigbee2MQTT process crashing due to a bug or out-of-memory.
- Manual restart of the Pi for maintenance.

After a coordinator restart, some or all sensors may show as "offline" in Zigbee2MQTT until they naturally wake up and reconnect.

Our fix: Ensure the Pi has a stable 5V 3A power supply. Set Zigbee2MQTT to start automatically on boot (systemd service). After a restart, wait 10–30 minutes for sleepy devices to reconnect on their own. If they don't, a quick power-cycle of the sensor (remove and reinsert battery) forces an immediate rejoin.

Official references

- [Zigbee2MQTT — Running as a systemd service](#)
- [Silicon Labs — End Devices: Parent-Child Relationship](#)

7. Parent Polling Timeout (Sleepy Device Eviction)

Likelihood: Medium

What happens

In Zigbee, battery-powered devices (called "sleepy end devices") do not stay connected to the network permanently. They go to sleep to save battery and periodically wake up to "poll" their parent (the coordinator, in our case) for any pending messages. The parent keeps a record of each child device and holds messages for it.

If a sleepy device fails to poll within the expected timeframe (usually 5–10 minutes of inactivity), the parent (coordinator) assumes the device is gone and removes it from its child table. When the device wakes up and tries to poll again, the parent does not recognize it — the device is effectively kicked off the network and must rejoin.

How this applies to us

All 6 of our sensors are sleepy end devices. Each one wakes up on its own schedule to send data or poll the coordinator. If the coordinator is busy (processing other tasks, experiencing high CPU load on the Pi) and misses the poll from a device, it might time out the child.

SONOFF sensors typically have a poll interval of 1–5 minutes. The coordinator's child timeout varies by firmware but is usually around 320 seconds (~5 minutes).

Our fix: Reduce the load on the Pi by ensuring no heavy processes compete with Zigbee2MQTT. Enable `last_seen: "ISO_8601"` in Zigbee2MQTT config to track when each device was last heard from, so you can spot poll timeouts early.

Official references

- [Silicon Labs — End Devices \(Polling & Parent Timeout\)](#)
- [Silicon Labs — Sleepy Devices Framework Guide](#)

8. Zigbee2MQTT Availability Check Marking Device Offline

Likelihood: Medium

What happens

Zigbee2MQTT has a built-in availability feature that pings devices periodically to see if they respond. If a device does not respond within a set timeout, Zigbee2MQTT marks it as "offline" or "unavailable." This is a software-level check — the device might still be connected to the Zigbee network but just not responding fast enough to the availability ping.

For battery-powered devices, this is especially problematic because they are asleep most of the time and cannot respond to pings. Zigbee2MQTT handles this by using the `last_seen` timestamp instead of active pinging — but only if configured correctly.

How this applies to us

If availability checking is enabled in our Zigbee2MQTT config but not tuned for battery devices, all 6 of our sensors could show as "offline" even though they are working fine — they simply cannot respond to pings because they are sleeping.

Our fix: In the Zigbee2MQTT configuration, enable availability with appropriate timeouts for passive (battery) devices. The default passive timeout is 1500 seconds (25 minutes). Set `last_seen` to "ISO_8601" so the backend can check when each device last reported.

Configuration example

In `/opt/zigbee2mqtt/data/configuration.yaml`:

```
availability: true
  active: { timeout: 10 }
  passive: { timeout: 1500 }
advanced:
  last_seen: "ISO_8601"
```

Official references

- [Zigbee2MQTT — Device Availability \(Official Docs\)](#)
- [Zigbee2MQTT — All Settings Reference](#)

9. Physical Obstructions and Metal Surfaces

Likelihood: Medium

What happens

Zigbee radio signals are weakened by physical obstructions. Different materials reduce signal strength by different amounts:

- Drywall/wood: 10–15% signal loss per wall.
- Brick/concrete: 25–40% signal loss per wall.
- Metal (appliances, mirrors, filing cabinets, steel doors): 40–60% signal loss or complete block.
- Water (fish tanks, water heaters): significant signal absorption.

A sensor placed behind or near a large metal object (like a refrigerator, metal door, or breaker panel) may lose its connection intermittently or completely.

How this applies to us

In the pilot home:

- The main_gate door sensor is on the front gate — which may be a metal gate in many Indian homes. Metal gates are very effective at blocking Zigbee signals.
- The kitchen_window sensor may be near metal kitchen appliances or a metal window frame.
- The safecall_button on the bedside table is usually in a clear area, so less affected.

Our fix: Relocate sensors slightly away from metal surfaces if possible. For the main_gate sensor on a metal gate, place the sensor on the wooden/concrete wall next to the gate, not on the gate itself. If range is still an issue, add a Zigbee router (mains-powered smart plug) near the problem area.

10. PAN ID Conflict (Nearby Zigbee Networks)

Likelihood: Low

What happens

Every Zigbee network has a unique identifier called a PAN ID (Personal Area Network ID). If two Zigbee networks in the same area happen to use the same PAN ID, devices can get confused about which network they belong to. They may try to join the wrong network, fail to communicate, or drop off entirely.

This is more common in apartment buildings or dense neighborhoods where multiple smart home setups might exist nearby.

How this applies to us

The pilot home is in a residential area. If any neighbor also runs a Zigbee network (smart bulbs, smart home hub, etc.) and their PAN ID conflicts with ours, our devices could experience intermittent connectivity issues.

Zigbee2MQTT auto-generates a PAN ID on first startup. You can check the current PAN ID in the Zigbee2MQTT dashboard (<http://192.168.1.14:8080>) under Settings → Advanced.

Our fix: If you suspect a PAN ID conflict, change the PAN ID in the Zigbee2MQTT configuration and re-pair all devices. This is rare in standalone homes but possible in apartments.

Official references

- [Zigbee2MQTT — Network Config \(PAN ID, Channel\)](#)
- [Silicon Labs — Zigbee Networking Concepts \(PAN ID\)](#)

11. Coordinator Firmware Issues

Likelihood: Low

What happens

The Zigbee coordinator (ZBDongle-E) runs its own firmware. If the firmware is outdated, buggy, or incompatible with the Zigbee2MQTT version, it can cause devices to drop off, fail to pair, or behave erratically. Firmware bugs can also cause the coordinator to run out of memory for its child table — meaning it literally has no room to track all the devices connected to it.

How this applies to us

Our ZBDongle-E uses the EFR32MG21 chip (Silicon Labs). The default firmware supports up to 20–30 direct children, which is well within our 6-device count. However, if the firmware is very old, there may be stability bugs that affect long-running connections.

Our fix: Check the current firmware version in the Zigbee2MQTT dashboard (Settings → About → Coordinator). Compare with the latest recommended version on the SONOFF or Zigbee2MQTT GitHub pages. Update firmware only if a specific issue matches a known bug in the release notes.

Official references

- [Zigbee2MQTT — Supported Adapters & Firmware](#)
- [SONOFF ZBDongle-E Product Page](#)

12. Network Key or Security Change

Likelihood: Low

What happens

Zigbee networks are secured with a network encryption key. Every device on the network knows this key. If the key changes (for example, by resetting Zigbee2MQTT, deleting its database, or reinstalling), all existing devices are suddenly unable to decrypt messages from the coordinator. They are locked out of their own network.

How this applies to us

This can happen if someone:

- Deletes the Zigbee2MQTT data directory (/opt/zigbee2mqtt/data/) and restarts — a new key is generated.
- Reinstalls Zigbee2MQTT without backing up the data directory.
- Changes the network_key setting in configuration.yaml.

The result is that all 6 sensors will appear offline and refuse to communicate. They must all be factory reset and re-paired.

Our fix: Never delete the Zigbee2MQTT data directory without a backup. The critical files are: configuration.yaml, coordinator_backup.json, database.db, and state.json. Back these up before any maintenance. If a key change happens accidentally, factory-reset each sensor and re-pair it.

Official references

- [Zigbee2MQTT — Configuration \(network_key\)](#)
- [Zigbee2MQTT — Backup & Restore](#)

13. permit_join Left Open (Constant Pairing Mode)

Likelihood: Low

What happens

When `permit_join` is set to `true` in the Zigbee2MQTT configuration, the network is permanently open for new devices to join. This means the coordinator is constantly listening for pairing requests, which consumes radio airtime and can interfere with normal device communication. It also poses a security risk — any Zigbee device in range could join the network.

How this applies to us

Our current configuration has `permit_join: true`. This was set during initial device pairing and never changed back to `false`. While all 6 devices are already paired, leaving this open is unnecessary and can contribute to instability.

Our fix: Set `permit_join: false` in `/opt/zigbee2mqtt/data/configuration.yaml` after all devices are paired. When you need to add a new device in the future, temporarily enable it through the Zigbee2MQTT dashboard (<http://192.168.1.14:8080>) — it has a button to allow joining for a limited time.

Official references

- [Zigbee2MQTT — Permit Join Configuration](#)

14. Sensor Firmware Bug or Hang

Likelihood: Low

What happens

Like any electronic device, a Zigbee sensor's own internal firmware can occasionally hang or crash. The radio stops transmitting, the device becomes unresponsive, and it appears offline. The device looks fine physically (no visible damage), but internally the processor is stuck.

How this applies to us

SONOFF Zigbee sensors are generally reliable, but any sensor running for months without a restart can theoretically experience a firmware hang. This is rare but does happen.

Our fix: Power-cycle the device: remove the battery, wait 10 seconds, reinsert it. The device will reboot and rejoin the network automatically. If the problem repeats frequently with the same device, it may be defective and should be replaced.

15. Quick Reference Summary

A quick-glance table of all disconnection reasons, their likelihood in our setup, and the fix:

#	Reason	Risk	Quick Fix	Affected Devices
1	WiFi / Zigbee Interference	High	Use Zigbee channel 25	All 6 sensors
2	USB 3.0 Port Interference	High	USB extension cable	All 6 sensors
3	Battery Low / Dead	High	Monitor & replace	All 6 sensors
4	Out of Range	Medium	Add Zigbee router / relocate Pi	main_gate, kitchen_window
5	Coordinator Crash	Medium	Stable power, auto-restart	All 6 sensors
6	Poll Timeout (Eviction)	Medium	Reduce Pi load	All 6 sensors
7	Z2M Availability Timeout	Medium	Configure passive timeout	All 6 sensors
8	Metal / Physical Blocks	Medium	Relocate sensor off metal	main_gate, windows
9	PAN ID Conflict	Low	Change PAN ID	All 6 sensors
10	Coordinator Firmware	Low	Check & update firmware	All 6 sensors
11	Network Key Change	Low	Backup data directory	All 6 sensors
12	permit_join Open	Low	Set to false	All 6 sensors
13	Sensor Firmware Hang	Low	Power-cycle sensor	Any individual sensor

End of Document